The Open Source Firefighter

Software Requirements & Development Roadmap

v1.0 · August 2026 · Public working document · OSF-4

*Document ID: **OSF-4** · one document of the Open Source Firefighter specification family.*

Andrew Morton · Palmer Lake, Colorado

This work is dedicated to Emily Barker, Nick Hutcherson, and Sydney Watson of the Rifle Helitack crew, killed in a burnover during initial attack on the Knowles Fire, western Colorado, June 27, 2026 — and to the two crewmates who survived it, and to all wildland firefighters who stand between fire and the rest of us.

They were the first to arrive. This system exists so that the ground they arrive on is already fighting back.

---

0. How to read this document

This is the software lane of the Open Source Firefighter. The specification documents are written to hold still; this one is written to move. Firmware architecture, language choice, libraries, and milestone ordering will all change as real implementations teach real lessons. **The specification is the contract; this is the plan.** Where they disagree, the specification and the register govern.

It is self-contained. A competent embedded developer — or an AI coding agent working from this text — should be able to read this alone, set up a development environment for the cost of two dev boards, and produce mergeable work without first reading the whole family. Where deeper authority is needed, this document names the source (the specification's vocabulary appendix for the wire vocabulary, the main specification for coordination behavior and the charter) rather than restating it.

Scope boundary, stated once: the software does not detect fire, locate fire, or aim anything at fire. It manages a shared water resource, coordinates turns among neighbors, reports honestly, and stays out of the operator's way. Detection events, where sensors produce them, are *reported* — never acted on offensively. This is a charter requirement, not a v1 simplification.

---

1. The problem, stated for software people

A node is a box on a wall with a controller, a low-power mesh radio, a few sensors, and a small number of motorized valves that put water on a roof and the ground around it. Many such boxes exist in a neighborhood. They share a water supply that cannot serve all of them at maximum draw simultaneously.

The software in each box must, on its own hardware and without any server:

1. **Know its own state** — supply pressure, zone positions, power condition, faults — and act on it.
2. **Announce itself** on the mesh so neighbors know it exists and what it can do.
3. **Negotiate a rotating schedule** so aggregate draw stays inside the supply while no structure goes uncovered too long during the ember-cast window.
4. **Obey a scoped, signed, expiring directive** from an incident commander when one arrives, and operate sanely when one never does.
5. **Degrade honestly** — radio gone, water gone, neighbor vanished, battery sagging — to a defined lower capability rather than an undefined failure.
6. **Take the operator's word over its own** wherever a physically present human touches it.
7. **Record what it did** so it can be verified by someone who does not trust the box.

The hardware is not the hard part and neither is the radio; both are commodity and both are solved. The hard part is (3), (5), and (7): distributed scheduling under partial information with nodes that come and go, degradation that stays predictable when assumptions break, and a record that means something without a trusted third party.

---

2. The layer model — a rough OSI stack

The build follows a rough OSI layering, deliberately. This is not academic tidiness; it is the mechanism that makes the system's central architectural promise enforceable. The coordination protocol must remain transport-independent — no revision may bind it to a specific physical, link, or network technology (main specification, coordination layer). Layering is how that promise is kept structurally rather than by good intentions: if coordination

logic cannot name a radio, it cannot depend on one.

OSI	OSF layer	What lives here	Module
1–2 Physical / Data link	**Physical & link**	Radio hardware, framing, the wire itself. LoRa today; anything tomorrow.	`hal/`, `link/`
3 Network	**Network**	Mesh addressing and relay. Meshtastic protocol at residential class; agency MANET at tactical class.	`link/`
4 Transport	**Envelope**	The compact binary envelope, fragmentation discipline, best-effort delivery semantics.	`proto/`
5–6 Session / Presentation	**Vocabulary & trust**	The closed verb enumeration, quantized field encoding, signing and verification, publication policy.	`proto/`, `policy/`
7 Application	**Coordination & operation**	Schedule negotiation, node state machine, degradation behavior, operator interface, ledger.	`coord/`, `state/`, `ui/`, `ledger/`, `app/`

The practical consequence: swapping LoRa for a different radio should touch layers 1–3 and nothing above. If a radio change requires editing `coord/`, the layering has been violated and the violation is the bug — not the radio.

Two departures from textbook OSI, both intentional:

- **The trust layer sits at 5–6, above transport, not bolted to the application.** Signing and publication policy are properties of the *boundary*, so they belong where every message must pass regardless of which application-layer feature produced it. This is why `policy/` is a chokepoint (§5).
- **There is no session state in the usual sense.** Delivery is best-effort by design; nodes exchange soft-state announcements with expiry rather than maintaining connections. A missed message is a normal event, not an error, and supply-pressure physics is the fallback coordination mechanism when messages do not arrive.

---

3. Fixed invariants — what software may not violate

From the charter and the specification. These are not implementation preferences; they are properties an implementation must preserve to be called compliant. Breaking one is a fork, not a contribution.

#	Invariant	Software consequence
I-1	**The system operates when fully isolated.** No mandatory external network, cloud, account, subscription, or license server for any core function. *(Charter Clause 2, absolute.)*	No boot-time network dependency. No feature behind a remote call. Provisioning is local. A node that has never seen the internet is fully functional.
I-2	**The inbound command vocabulary is closed and enumerated.**	A verb outside the enum has no representation and cannot be constructed, transported, or acted on. Enforced at the type level, not by validation-after-parse.
I-3	**Translation is outbound-only, at the boundary.**	The mesh never carries a rich external format. CoT/CAP/Contact-ID rendering happens at a gateway, above coordination, never between nodes.
I-4	**The operator physically present prevails.** *(Charter Clause 7.)*	No software configuration may disable an operator control. No automated lockout of an operator's own resources. Conflicts at the point of use resolve to the operator — in code, not in

		documentation.
I-5	**No autonomous suppression or targeting.**	The software never decides what to attack. It decides when it is this node's turn to wet its own assigned coverage.
I-6	**Transport independence.**	Coordination imports an interface, never a radio. See §2.
I-7	**Failure degrades to baseline, never to worse-than-nothing.**	Every failure path has a defined, tested destination state. Loss of power drives auto-return valves closed by physics; software must not fight it or assume it can prevent it.
I-8	**The record is verifiable without trusting the node.**	Ledger entries signed and chained. A node that lies must be detectable by someone holding only the published chain.

4. Compliance boundaries to design against

Read this before you build, not after. Nothing here is difficult to accommodate if you know about it early; several items are expensive or impossible to retrofit. This section exists because an implementer can otherwise spend months on a design that is unshippable for a reason no software person would think to check.

Orientation, not legal advice. Requirements vary by jurisdiction and change between editions. Per the family's adoption-by-reference discipline, verify against the issuing authority's current edition for your location — this document deliberately does not restate values that will age.

4.1 The three traps that bite hardest

1. Do not accidentally become a fire alarm system. This is the big one. A device that detects fire conditions and reports them to a monitoring station is, in most jurisdictions, a *fire alarm system* — which pulls in NFPA 72, product listing (UL 864 class), monitoring-station standards, and an entirely different regulatory posture with certification costs to match. OSF sits deliberately outside that boundary: it is a water-application resource-coordination system that reports its own operational state. **The distinction to protect in software:** the system reports *what it is doing* (zone open, supply degraded, node in distress), not *that a building is on fire*. Detection events, where present, are situational telemetry for coordination and command awareness — not alarm signals to a monitoring authority. If a feature request pushes toward "notify the alarm company that there is a fire," that request changes the product's regulatory class. Escalate it; do not implement it quietly.

2. Cross-connection and backflow. The node connects to a potable supply and can create a path from irrigation-side plumbing back toward the main. Backflow prevention and cross-connection control are enforced by water purveyors and plumbing codes essentially everywhere, and a water district can refuse an installation over it. This is primarily a hardware and installation matter (Annex A), but it lands in software one specific way: **do not design control logic that assumes it can be the safety barrier.** Backflow protection is a physical device. Software must never be positioned as the thing preventing contamination, and no failure mode may depend on the controller behaving correctly to protect the potable supply.

3. Radio equipment authorization. The mesh radio is an intentional radiator. In the United States, equipment marketed with an intentional radiator requires FCC equipment authorization; elsewhere the equivalent regime applies. **The practical path is a pre-certified radio module, unmodified in its RF characteristics** — changing transmit power, antenna, or duty cycle beyond the module's certified conditions can void the certification of the finished device. In software: power and duty-cycle limits are not tunables to optimize freely. Treat the certified operating envelope as a constraint the code respects, and keep region-dependent parameters in one clearly marked place (§4.2).

4.2 Radio and spectrum

- **Band and power follow the jurisdiction, not the specification.** The reference implementation uses the North American 902–928 MHz ISM allocation; other regions use their own (863–870 MHz in Europe, for example) at their own permitted power. Some regions impose **duty-cycle limits that materially affect protocol design** — a heartbeat cadence legal in one region may not be in another. Build region parameters as configuration with enforced ceilings, not constants scattered through the code.

- **Stay off public-safety spectrum.** Interoperating with agency radio happens at the network interface (Ethernet/IP at a gateway), not by transmitting on public-safety frequencies. Licensed-band operation is a different regulatory world and is not part of this design.
- **Stay off amateur spectrum, and understand why.** The trust layer signs every outbound message. Encryption and obscured messaging are prohibited on amateur allocations — in the United States under FCC Part 97.113(a)(4) — and amateur operators must identify by call sign at intervals. A signing node and the amateur service are therefore mutually exclusive by regulation. Do not build an amateur-band mode. If a downstream implementation wants one, it must disable signing, and a node with signing disabled is not an OSF node.
- **Tactical-class radios are their own regime.** Agency MANET radios may carry export-control and procurement restrictions. The node integrates with them at the data interface and does not embed them.

4.3 Electrical and installation

- **Industrial control panel and enclosure practice** (UL 508A class listing, NEC/NFPA 70 wiring, enclosure type ratings) governs the box. This is Annex A territory; the software consequence is small but real: **do not create a design that requires the panel to be opened during normal operation**, and do not assume access to internals a listing would require to stay closed.
- **Low-voltage control is where the software lives.** Keeping control and communication in the low-voltage domain keeps most electrical compliance in the hardware lane, where it belongs.

4.4 Interfacing with government and public-safety systems

These apply when a deployment publishes into an official system — not to a standalone neighborhood domain, which is the common case.

- **Public alerting is authorization-controlled.** CAP is the format for public alerts, but **the ability to originate a public alert into a national alerting system is granted, not assumed** — origination authority is held by qualified authorities under a formal process. A node or gateway may *produce* CAP-formatted output; it may not self-authorize into an alerting network. Design the boundary so an authorized operator's system consumes OSF output, rather than OSF injecting into an alerting network directly.
- **Incident-command interoperability follows NIMS/ICS conventions.** Where the system presents information to incident command, it should fit existing command structure and terminology rather than inventing a parallel one. The tactical picture publishes as CoT.
- **Federal systems bring federal requirements.** If a deployment ever hands data to a federal system, expect requirements around cryptographic module validation (FIPS-class), information-security controls, records handling, and accessibility for any government-facing interface. Do not build toward this speculatively — but **choose cryptographic primitives that have validated implementations available**, so the door stays open. Selecting an exotic cipher because it is elegant closes it permanently.
- **Records and evidence.** The ledger is designed to be evidentiary (I-8). Where output may become part of an official record — a utility's mitigation record, an after-action report — integrity and retention matter more than convenience.

4.5 Privacy and data

- **The node has no camera and no recording microphone by design**, and that is a compliance asset, not a limitation. Imagery and audio recording pull in privacy law, retention obligations, and consent questions a fire-defense appliance has no reason to accept. Acoustic sensing, where used, produces *derived state* (machinery present, engine running), never a recording. **Do not add an imagery or audio-capture path.** If one is ever proposed it is a specification change with a compliance review, not a feature.
- **Position data is coarse by design** — adequate for coordination and command, not precise tracking of people.
- **Detection events report source, bearing class, and confidence class** — deliberately minimal, deliberately not personal.

4.6 Licence inheritance from upstream dependencies

A dependency's licence is a constraint you inherit, not a choice you make. The reference mesh implementation is Meshtastic, whose firmware is distributed under **GPLv3**. Firmware that links into, embeds, or derives from that codebase is GPLv3 by operation of that licence, and no downstream decision changes it.

The boundary is worth stating precisely, because it is narrower than it first appears:

- **Captured:** firmware built on or linked against the upstream mesh implementation.
- **Not captured:** the OSF specification documents themselves, the coordination protocol, the closed vocabulary, and the envelope format. These are specifications. A specification describing what a message contains is not a derivative work of any program that happens to carry that message. A clean-room implementation written from the specification alone inherits nothing.

Two practical consequences. **Check the licence of every dependency before adopting it**, and prefer dependencies whose terms are compatible with the project's own licence selection. And **keep the protocol implementation separable from the transport implementation** — which the layer model (§2) already requires for transport independence, and which turns out to serve a licensing purpose as well. A coordination layer that cannot name a radio also cannot inherit that radio's licence.

Upstream trademark terms are a separate matter from upstream licence terms and bind independently: a project may be free to use the code and still not free to use the name.

4.7 The rule of thumb

If a proposed feature would make the system (a) tell an outside authority that there is a fire, (b) originate a public alert on its own authority, (c) record imagery or audio, (d) act on the supply in a way that only software prevents from contaminating it, or (e) transmit outside the radio module's certified envelope — **stop and escalate.** Each changes what the device legally is. None are necessary to do the job.

---

5. Decomposition — the code map

The layering below mirrors §2, which is what makes the invariants enforceable by structure rather than discipline. Names are suggestions; boundaries are not.

Module	Responsibility
`app/`	Main loop, task scheduling, wiring the layers together
`ui/`	Operator inputs (debounce, deliberate-press timing) and indicator state
`policy/`	Publication policy — the trust boundary. Nothing leaves except through here
`ledger/`	Signed, chained snapshots
`coord/`	Schedule negotiation, neighbour table, domain view
`proto/`	Envelope, codec, verb dispatch, vocabulary definitions generated from Appendix C
`link/`	Transport interface — one implementation per radio, one for simulation
`state/`	Node state machine, timers, fault model
`hal/`	Hardware abstraction: valves, sensors, power, controls, indicators — with a real implementation and a simulated one

Module names are a suggestion. The boundaries between them are not.

Dependencies point downward only. `coord/` may call `proto/` and `state/`; `proto/` may not know `coord/` exists. `hal/` knows nothing above it. This is what makes I-6 structural.

`policy/` is a chokepoint, not a library. Every outbound byte passes through it. It is the only module permitted to emit. If a developer can transmit without traversing `policy/`, that is a defect of the highest severity — it means the trust layer is advisory, and an advisory trust layer is not one.

---

6. The development environment — start with two dev boards

The full physical node involves plumbing, valves, an enclosure, and a wall. None of that is needed to write, run, or test the software. Valves and flow sensors are trivially simulated; the interesting behavior is entirely in scheduling, degradation, and protocol.

6.1 The simulation contract

Everything physical sits behind a hardware abstraction layer with two implementations. This is the single most important architectural decision in the project, because it is what lets a contributor with no hardware do real work.

The interface is deliberately narrow. It exposes, and exposes only: **zone position** as read from independent feedback rather than from commanded state; **zone command**; raw readings for **supply pressure**, **reservoir level** where fitted, and **flow**; **power condition** (mains, battery, low, critical); the state of each **operator control**; **indicator output**; and a **monotonic clock**. Nothing above `hal/` may reach the hardware by any other route.

Two implementations satisfy it. One drives real hardware. The other simulates it — and the simulator is where most development happens, so it deserves as much care as the driver.

The simulated implementation should be a small physical model, not a stub: a valve that takes realistic seconds to travel and *can fail mid-travel*; a supply whose pressure sags when simulated neighbours draw; a battery that discharges; a flow sensor reading zero on a kinked line.

The simulator should be able to lie and to break. A simulator that models only the happy path produces software that works only on the happy path. `hal_sim/` supports scripted faults — valve seizes, feedback disagrees with command, pressure collapses, battery falls off a cliff, sensor reads implausible values — because embedded failure is the credibility mechanism of this project (§14) and belongs in the harness, not only in the prose.

6.2 Two build targets from day one

- **Host build (native).** Runs on a laptop. Many simulated nodes in one process or several, over a simulated link with configurable loss and latency. Ninety percent of development happens here: fast iteration, real debuggers, property tests, fifty-node domains.
- **Device build (ESP32).** Same source, `hal_real/` and the LoRa link. Where timing, memory, power, and radio reality get checked.

CI runs the host build on every commit; the device build on a cadence.

6.3 Minimum development hardware

Purpose	Item	Note
Node compute	Any ESP32 dev board	Two is meaningfully better than one — a mesh cannot be tested with a single radio.
Radio	LoRa module for the local ISM allocation	Combined ESP32+LoRa boards are ideal. Band and power follow the jurisdiction (§4.2).
Everything else	*(nothing)*	Valves, sensors, pumps, enclosure, plumbing: simulated.

A contributor who owns nothing can still work productively on the host build alone.

6.4 Language and toolchain

Not yet fixed, deliberately — this should be decided and defended by the first serious implementer, not inherited from a document. Constraints the choice must satisfy:

- Runs on ESP32-class hardware within its memory budget.
- Compiles for device and host without source forks.
- Has a workable property-based testing story (the §3 invariants want property tests, not examples).
- Approachable enough that a competent hobbyist can contribute — the adoption model depends on it.
- **Has validated cryptographic implementations available** (§4.4).

C or C++ with ESP-IDF is the conservative answer. Rust is credible and buys real safety. MicroPython lowers the contribution barrier at a cost in determinism. **Pick one, write the reasoning down, make it a decision of record.**

---

7. The node state machine

State	Meaning	Leaves when
`BOOT`	Power-on, self-test, hardware inventory	Self-test completes → `IDLE`, or fault → `FAULT`
`IDLE`	Armed and quiet. Heartbeating. Valves closed.	Activation (command, local, or schedule) → `ACTIVE`; disarm → `DISARMED`
`DISARMED`	Operator has locked out automatic operation. Still heartbeats and reports.	Operator re-arms → `IDLE`
`ACTIVE`	Protection running. Executing this node's slot.	Deactivation, expiry, or supply loss → `IDLE/DEGRADED`
`YIELDED`	Deliberately holding off — another node's turn, or a directive said cease.	Slot arrives, or directive expires → `ACTIVE/IDLE`
`DEGRADED`	Reduced capability. Supply gone, zone failed, or battery low.	Condition clears → prior state; worsens → `REPEATER`
`REPEATER`	No water capability. Still carries mesh traffic.	Water capability restored → `IDLE`
`ISOLATED`	No mesh contact within timeout. Autonomous conservative operation.	Neighbor heard → rejoin coordination
`DISTRESS`	Operator signaled distress. Latched full protection, status published.	Operator clears
`FAULT`	Cannot perform core function; reporting the reason.	Fault cleared or serviced

`ISOLATED` is not an error state. It is a normal operating mode with conservative defaults, and it must be *good*. Most nodes will spend most of their lives with intermittent mesh contact. A node that panics when alone is a broken node.

`REPEATER` is the graceful-degradation showcase. A node whose water has failed is still worth having on the wall because it carries its neighbors' traffic.

`DISTRESS` latches. It does not time out; a human clears it.

Every transition is a ledger event (§13).

---

8. The wire protocol and vocabulary

the specification's vocabulary appendix (Appendix C) is the authority for field layouts, quantization, and byte-level examples. The vocabulary itself is reproduced here because it is the data dictionary, the dispatch switch, and most of the software design — a developer should not need a second document open to know what the system can say.

8.1 Envelope constraints

The wire format is a compact binary envelope sized to the most constrained transport in the family. LoRa mesh payloads live in the ~200-byte range. The signature is the large field (Ed25519 class, 64 bytes) and is non-negotiable — the trust layer rides in the envelope. **Realistic body budget after header and signature: 60–100 bytes.** That number is the discipline; every verb must fit it.

Readings on the wire are **single-byte quantized classes**, not precise values: coordination needs "pressure sagging," not 47.3 psi. Full-resolution values stay in the local log.

8.2 The V1 verb list

Group	Verbs	Notes
Presence	`HEARTBEAT`	status flags, battery class, supply class, capability bits, coarse position
State	`ARM`, `DISARM`, `ACTIVATE`, `DEACTIVATE`, `ZONE_OPEN`, `ZONE_CLOSE`, `FAULT`, `MAINT_REQ`	`FAULT` carries a fault-code enum

Supply	`PRESSURE`, `FLOW`, `LEVEL`	quantized single-byte classes
Coordination	`SCHED_STATE`, `SCHED_PROPOSE`, `YIELD`, `RESUME`	schedule negotiation per main specification
Command	`CEASE_OPS`, `RESUME_OPS`, `PRIORITY_PROTECT`, `BROADCAST_ALERT`	scope filter, authority, expiry; short, audited vocabulary
Safety	`DISTRESS`	node, position, timestamp — deliberately minimal
Events	`DETECT`, `ENGINE_STATE`, `ENERGY`	detection events (source / bearing class / confidence class); acoustic-derived machinery state; power transitions
Ledger	`SNAPSHOT_COMMIT`	hash chain link; the payload is the commitment

Roughly two dozen verbs. **The vocabulary is closed and short.** Additions go through charter governance; smallness is a protected property of the protocol, not a v1 limitation.

Capability bits ride in the heartbeat (supply class, stored water, pump control, detection payload, gateway role). This is how the domain knows what each node is without a registry, and how directive scope filters resolve locally. Get this right early; much depends on it.

8.3 Implementation requirements

- **Generate, do not transcribe.** The Appendix C enum table should compile to the header mechanically. Hand-copying a wire vocabulary is how two implementations silently diverge.
- **The verb enum is the dispatch switch.** A `default:` case doing anything but count-and-drop is a bug.
- **Assert the byte budget in tests.** A verb exceeding its maximum encoded size must fail at build or test time, not on a congested mesh at 0300.
- **Round-trip fuzz everything.** Encode/decode identity for all verbs, plus decode-arbitrary-bytes-without-crashing. The radio will deliver corrupt frames; the parser must be boring about it.
- **Golden vectors.** Appendix C specifies byte-level examples for `HEARTBEAT`, `ZONE_OPEN`, `CEASE_OPS`, `DISTRESS`. These are the conformance fixtures a second implementation tests against first.

8.4 Coverage math the node computes

Coverage and supply planning is not incidental to the software — a node self-calibrates its delivered supply at commissioning, sizes its zones against it, and reports coverage capability in its capability bits. Implementers should not have to go find these relations. They are reproduced here for the same reason the verb list is: they are the working data dictionary of this part of the system.

The Supply & Coverage Planning Table (OSF-2, Appendix A) is the authority for planning figures, head classes, and derated field values. This section carries only the relations a node computes.

Quantity	Relation	Notes
Effective radius at pressure	`r ≈ r_design × √(P / P_design)`	Design pressure for structure-protection heads is 50 psi. Below ~40 psi the radius collapses and derived spacing no longer overlaps.
Flow at pressure	`q ≈ q_design × √(P / P_design)`	Same square-root pressure relation.
Head-to-head spacing	`S = r`	The baseline. Spacing equals *radius*, never diameter.
Wind-derated spacing	`S = 0.9 × r` (hard wind `0.8 × r`)	Most fire days are wind days; derate by default.
Precipitation rate	`PR = 96.25 × GPM ÷ S²`	Inches per hour, square spacing, GPM per head. Published head data assumes zero wind.
Time to target depth	`t = (depth ÷ PR) × 60`	Minutes. Planning targets are light dampening through pre-front soak.

Sector wetted area	`A = (arc ÷ 360) × π × r²`	For arc-limited heads; the basis for coverage-fraction calculations.
Stored-water duration	`t = tank_gallons ÷ zone_GPM`	The number that makes rotation necessary rather than optional.

Two implementation notes with teeth. **A node must not report coverage it cannot deliver** — computed coverage is derived from *measured* supply at commissioning, never from the planning figures, and where the two disagree the measurement governs. And **the arc matters to the numbers**: an arc-limited head applies its full flow to a fraction of the circle, so precipitation rate rises as arc narrows. A coverage calculation that ignores arc will under-report application rate on exactly the part-circle heads the placement doctrine prefers.

---

9. Coordination — the scheduling problem

This is the invention, and the most interesting code in the project.

The problem: N nodes share a supply sustaining fewer than N at full draw. Each needs its coverage wetted often enough that no structure has an extended dry gap during the ember-cast window. Nodes join and leave. Messages get lost. There is no coordinator and there must never be one.

Inputs, arriving via heartbeat and coordination verbs: each node's supply class, current supply state, coverage pattern, activation state, battery condition.

Output: each node's own decision about whether it runs right now. **Not** a global schedule computed and distributed — a local decision that, taken by every node from shared information, produces a sane global pattern.

Properties the implementation must have (these are the acceptance tests):

1. **Supply safety.** Aggregate committed draw stays within sustainable capacity. Violating this is the one unforgivable failure — it empties the tank and defeats everyone.
2. **Coverage floor.** No participating structure exceeds a maximum dry interval while the domain is active.
3. **Convergence under churn.** Nodes appearing, vanishing, reappearing must not produce oscillation, thrash, or a stuck domain.
4. **Graceful partial participation.** A domain where only some structures have nodes still works for the participants — the honest and common case.
5. **No coordinator, no election.** Leader election is a failure mode waiting for a fire. Prefer algorithms needing no leader at all.
6. **Physics as backstop.** A node that misses coordination and draws anyway is answered by pressure sag and must yield to it. Falling back to hydraulic reality is designed behavior, not accident.

In one embodiment, the schedule is computed as follows (this is one method of satisfying the properties above, not the only one): each node computes a deterministic priority from its identity, supply class, and time-since-last-wetted, seeded so all nodes computing from the same neighbor table reach the same ordering. Nodes take slots in that order, sized to the supply budget. Time-since-wetted rising monotonically prevents starvation. Stored-water nodes bias toward running during municipal pressure sags, since they are not competing for the same resource. Deterministic-from-shared-state means convergence without negotiation traffic in the common case; `SCHED_PROPOSE`/`YIELD` handle disagreements.

Build this in simulation first, fifty nodes on a lossy link. Far cheaper to find an oscillation with a hundred simulated nodes than three real ones on a bench.

---

10. Trust layer and publication policy

`policy/` implements the boundary between what the node knows and what the world sees.

Outbound: nothing transmits except through this module. It is where the enumerated-message discipline becomes real, where signing happens, and where — at a gateway only — rendering into CoT, CAP, or Contact-ID occurs. A non-gateway node never speaks an external format.

Inbound: commands are verified before they are anything else. A directive carries issuing authority, scope filter, and expiry. The implementation must:

- **Verify the signature** before the message influences any state.
- **Resolve scope locally** against this node's capability bits — no registry lookup, no asking permission.
- **Honor expiry and resume automatically on it.** A directive not refreshed dies. *A stale silencing order must not*

outlive the attention of the commander who issued it. This is a safety property; implement the timer before the directive handling.

- **Never allow an inbound message to disable an operator control** (I-4). A directive constrains automated behavior; it does not reach the panel.

Key handling is an open design question (§17), not a solved one. What is settled: verification must be possible offline by a node that has never contacted any authority server (I-1), and primitives should have validated implementations available (§4.4).

---

11. The degradation ladder

Each rung is a defined behavior with a test. This table *is* the graceful-degradation architecture in software terms.

Failure	Node behavior	Domain behavior
Mesh lost	→ `ISOLATED`; conservative autonomous cycle	Remaining nodes reschedule around the absence
Neighbor vanishes	Drop from neighbor table after timeout	Reschedule; no stall waiting for the missing
Supply pressure sags	Yield draw; report `PRESSURE`	Others sense the same and back off; physics coordinates
Water supply fails	→ `DEGRADED`, then `REPEATER`	Node keeps carrying traffic; coverage gap reported
One zone fails (clog, kink, seized valve)	Report `FAULT` with code; continue on remaining zones	Coverage map updates; neighbors may compensate
Valve feedback disagrees with command	Trust feedback, report fault	Never report protection that is not physically happening
Battery low	Shed load: reduce cycle, prioritize reporting over wetting	Domain sees reduced capability in heartbeat
Power lost	Auto-return valves close by physics; nothing to do	Node disappears; domain reschedules
Directive missed	Continue local operation, yield via pressure	Best-effort delivery is the design, not a defect
Controller crash	Watchdog reset → `BOOT` → self-test	Brief absence, then rejoin

Design rule: every one of these degrades to *the unprotected baseline the world already accepts*, and preferably a bit better. No failure may produce a state worse than having no system at all. A proposed feature that can fail into worse-than-nothing does not belong in the node.

---

12. Operator interface

Per the capability requirements (specification and Inspector’s Draft §3.4): a physically present operator can initiate protection, lock out automation, command each zone independently, and signal distress — in structural gloves, in darkness, without instruction, without a personal device, and without any network or account.

Software requirements regardless of the physical control form:

- **Debounce everything**, then apply intent timing. The distress signal requires a deliberate press of roughly three to five seconds; brush contact and bumps must not trigger it.
- **Controls are inputs the controller reads.** They are not emergency-stop devices and no safety category is claimed. The hardware kill is the disconnect. Never write software as though a button bypasses it.
- **Indicators report physical reality, not intent.** A zone lamp shows *valve feedback*, not *commanded state*. If those disagree, the lamp tells the truth and a fault is raised. This single rule prevents the most dangerous class of lie the system could tell.
- **State readable at a glance without a display:** power, radio link, readiness, fault, active, distress.
- **No software configuration may disable an operator control** (I-4). There is no settings flag for this, and there must not be a code path that could grow one.

---

13. The ledger

Signed, chained snapshots — `SNAPSHOT_COMMIT`, where the payload is the commitment.

- Every state transition, directive received, zone command, and fault is an event.
- Events batch into periodic snapshots; each snapshot commits a hash chaining to the previous.
- The chain makes an after-action report evidentiary rather than narrative — a record whose integrity does not depend on the credibility of whoever holds the box.
- **Complement, not substitute, for physical verification.** The ledger attests what was *commanded*; a thermal overflight attests what physically *happened* (Annex A). Make the two easy to correlate: timestamps and zone identities that line up with an external observation.

Implementation notes: flash is small and write cycles finite. Snapshot cadence is a tunable. Chain verification must be possible from the published chain alone, by a third party with no access to the node.

---

14. Test strategy

Property tests for the invariants. The §3 table is a test list. *No path leaves valves commanded open through a power-loss sequence. `DISARM` is reachable from every state. An unenumerated verb cannot be constructed. Nothing transmits without traversing `policy/`.* These are properties, not examples.

Conformance fixtures. The Appendix C golden vectors plus a growing corpus of encoded messages. The goal is the interoperability property: *two independent implementers working from the vocabulary table produce interoperable nodes without coordination.* That claim needs a test suite to be true.

Scenario tests — the project's unusual asset. The Deadman Fire scenario (OSF-2.1) is a complete, physically honest, hour-by-hour operational narrative with realistic embedded failures. **It should be encoded as an automated integration test:** a simulated domain matching its geometry, driven through its timeline, with its failures injected at the right moments — the seized valve, the flat battery, the burnt-through hose, the missed directive — and assertions that the domain behaves as the scenario describes. When the software passes Deadman, it has been tested against a scenario a fire chief already read and called realistic.

Fault injection is mandatory. Per §6.1 the simulator must break. A suite that exercises only success has verified nothing about a system whose credibility rests on failing gracefully.

---

15. Roadmap

Ordered by dependency, sized to be individually completable, each with an acceptance criterion that can be demonstrated rather than argued.

#	Milestone	Acceptance criterion
M0	**Skeleton and simulation harness.** Repo, host build, CI, `hal/` interface with `hal_sim/`, scriptable faults.	`make test` runs on a laptop with zero hardware. A simulated node boots and logs a main loop. Faults injectable from a script.
M1	**State machine.** §7 implemented with timers and fault model. No radio.	Property tests pass for the §3 invariants. Every state reachable and exitable in simulation.
M2	**Wire format.** `proto/` generated from Appendix C; envelope, codec, dispatch.	Golden vectors for `HEARTBEAT`, `ZONE_OPEN`, `CEASE_OPS`, `DISTRESS` match the specification's vocabulary appendix byte-for-byte. Round-trip fuzz clean. Byte budget asserted in CI.
M3	**First real hardware.** ESP32 + LoRa, `hal_real/` with simulated plumbing.	Two physical boards exchange heartbeats and decode each other's capability bits.
M4	**Coordination.** §9 algorithm in simulation at scale, then on real radios.	50 simulated nodes on a lossy link: supply budget never exceeded, coverage floor held, converges under churn. Then three real nodes rotating correctly.
M5	**Trust layer.** Signing, verification,	Unsigned and expired directives

	`policy/` chokepoint, command broadcast with scope and expiry.	rejected. Scope resolves locally. Automated test proves nothing transmits outside `policy/`. Expiry auto-resumes.
M6	**Operator interface and degradation.** §11 ladder and §12 interface complete.	Every rung of the degradation table demonstrated with injected faults. Indicators track feedback, not command.
M7	**Ledger and event interface.** Chained snapshots; outbound rendering at a gateway.	Tampering with any historical event detectable from the published chain. An external consumer reads the domain picture.
M8	**Deadman as integration test.** The scenario encoded and passing.	The simulated domain runs the Deadman timeline with its failures and behaves as written.
M9	**Conformance and second implementation.** Published test suite; an independent implementation interoperating.	Two implementations, written from Appendix C without coordinating, exchange traffic and coordinate a shared schedule.

M0 through M2 are a solo weekend-scale effort for someone competent and require no hardware. That is deliberate: the barrier to a first useful contribution should be a laptop and an afternoon.

M4 is the research risk. Everything before it is engineering; the scheduling algorithm is where genuine design work lives. Budget accordingly and expect to throw the first one away.

M9 is the real finish line for the software lane, because it proves the claim the whole specification rests on: that the documents are sufficient for independent implementation.

---

16. Getting started

1. **Read this document, then the specification's vocabulary appendix (Appendix C).** Together they are enough for M0–M2.
2. **Skim §4 before you design anything.** The compliance boundaries are cheap to respect early and expensive to retrofit.
3. **Skim the main specification's coordination sections** before M4; the Deadman scenario before M8, or earlier if you want to understand what you are building.
4. **Do not start with the radio.** Start with `hal/` and the state machine on the host build. Radio-first is the most common way embedded projects stall.
5. **Do not start by optimizing.** The node has enormous time budgets by software standards — a valve takes seconds to travel, a schedule slot is minutes. Clarity beats cleverness everywhere in this codebase.
6. **Write the decision down when you make one.** Language, scheduling algorithm, key handling, snapshot cadence — these are decisions of record. The next implementer's ability to work depends on knowing why, not just what.

---

17. Open questions

Unresolved. A contributor who resolves one well has made a real contribution.

- **Language and toolchain** (§6.4). Needs a decision and a written rationale.
- **Key management and identity.** How does a node get an identity a neighbor will trust — offline, with no authority server, in a community where units are bought at different times by different people?
- **Command authority.** How does a node know an incident commander's signature is an incident commander's, without a registry it cannot reach? *In one embodiment,* the authority public key for an incident is distributed with the incident communications plan by the same means as radio frequency assignments — the analogue of an ICS-205 — and is accepted by nodes for the duration of the incident, bounded by the directive expiry already required by the trust layer. *In a further embodiment,* a domain designates one or more steward keys at commissioning, and an incident authority key is accepted when countersigned by a steward, so that a node can verify command authority entirely offline against material it already holds.
- **Scheduling algorithm** (§9). The deterministic-priority method described in §9 is disclosed as one embodiment

and is expected to be refined or replaced as implementations gather field data; the properties it must satisfy, listed there, are the requirement.

- **Time.** Coordination and expiry need a shared-enough clock. GNSS where present — but what about a node without it, in a valley, after a week?
- **Snapshot cadence and flash budget.** Ledger granularity against device lifetime.
- **Configuration and commissioning.** Local-only by charter — but what is the actual ritual for a homeowner, in gloves, at dusk?

---

18. Governance

The protected core — coordination protocol, closed vocabulary, trust layer — may not be bound to any single transport, vendor, or mandatory external dependency by any revision. Vocabulary additions go through charter governance; smallness is a protected property of the protocol.

Contributions that implement this document are welcome without ceremony. Contributions that change what the software *must do* are specification changes and belong in the register, not in a commit.

The community infrastructure this project needs — the change-proposal process, the build registry, the conformance claim, the safety-defect channel, and the library of known-good configurations that is the real goal — is described in the Public Roadmap (OSF-3.2, §9, The Development Commons). Software contributors should read it before the first substantial pull request, because where a decision gets recorded matters as much as what the decision was.

This document is expected to change faster than any other in the family. That is its purpose. The specification holds still so that this one can move.